

Лабораторная работа 7

Обработка ошибок и модули

Введение

Ошибки (исключения) — неотъемлемая часть работы программ. Python предоставляет механизм try-except для их перехвата и корректной обработки. Модули расширяют возможности языка, подключая библиотеки с готовыми функциями.

Цель работы: научиться обрабатывать исключения, использовать встроенные модули и создавать собственные.

Теоретическая часть

Блок try содержит код, который может вызвать ошибку. Блоки except перехватывают конкретные типы исключений.

else выполняется, если ошибок не было, finally — в любом случае.

Импорт модулей: `import module`, `from module import name`, `import module as alias`.

Стандартная библиотека: `math`, `random`, `datetime`, `os`, `sys` и другие.

Задания

1. Напишите функцию `input_int(prompt)`, которая запрашивает целое число до тех пор, пока не будет введено корректное значение (перехват `ValueError`). Используйте её в программе, вычисляющей частное двух чисел с защитой от деления на ноль.
2. Создайте генератор паролей, удовлетворяющий требованиям: длина не менее 4, хотя бы одна цифра, одна заглавная буква и спецсимвол. Используйте модули `random` и `string`. При длине `<4` выбрасывайте собственное исключение `raise ValueError`.
3. Напишите программу, которая запрашивает путь к файлу и с помощью `os.path` выводит: существует ли он, размер (для файла), является ли файлом или директорией. Корректно обработайте отсутствие файла.
4. Модифицируйте безопасный ввод числа так, чтобы все ошибочные вводы записывались в `error.log` с отметкой времени (используйте `datetime`).
5. Пользователь вводит две даты в формате ГГГГ-ММ-ДД. Программа вычисляет количество дней между ними с помощью модуля `datetime`.
6. Создайте модуль `mymath.py`, содержащий функцию `gcd(a, b)` (наибольший общий делитель). Импортируйте его и протестируйте.
7. Напишите скрипт, который с помощью `sys.argv` принимает два аргумента (имя файла и слово) и подсчитывает, сколько раз это слово встречается в файле.
8. Обработайте ситуацию, когда при открытии файла возникает `FileNotFoundError`. Выведите понятное сообщение и предложите пользователю ввести имя заново.
9. Создайте собственную иерархию исключений: `MyError` (базовый) и `MyValueError`, `MyTypeError`. Продемонстрируйте их выбрасывание и перехват.
10. Напишите программу, которая проверяет корректность e-mail адреса (простая проверка: наличие `@` и точки после неё) и выбрасывает исключение при неверном формате.
11. Используя модуль `platform`, выведите информацию о системе (ОС, версия Python).

12. Реализуйте функцию `safe_division`, которая возвращает результат деления или бесконечность (`float('inf')`) при делении на ноль, с занесением попытки в лог.
13. Напишите программу, которая с помощью `time.sleep` делает паузу между выводом чисел от 10 до 0 (обратный отсчёт).
14. Создайте декоратор (без углублённого объяснения) `retry`, который повторяет функцию до 3 раз при возникновении исключения.
15. Используя модуль `argparse`, напишите программу, которая принимает аргументы командной строки: `--name` и `--count`, и выводит приветствие `name` `count` раз.

Контрольные вопросы

1. Для чего нужна конструкция `try-except`?
2. Чем отличается `except Exception` от `except ValueError`?
3. Что делает блок `finally`?
4. Как импортировать только одну функцию из модуля?
5. Какие модули из стандартной библиотеки вы знаете?

Выводы

Освоена обработка исключений, работа со стандартными модулями. Программы стали устойчивее к некорректным данным.